Encore Moment

PC-Based Audience Reaction Capture

Group Encore

Chul Park
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
parkchul1@hanyang.ac.kr

Pavel Kim
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
skydish@hanyang.ac.kr

Soyeon Seol
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
ssy050915@hanyang.ac.kr

Yujeong Heo
Department of Information Systems
Hanyang University
Seoul, Republic of Korea
gjdbwjd03@hanyang.ac.kr

***Abstract*—Encore Moment captures audience reactions as reviewable viewing memories. Its first implementation is a PC application connected to a team-controlled test player and an AWS inference server. Playback-reference echo cancellation supplies audience audio for laughter, scream, and gasp recognition. Server-side audio and facial analysis produce events with observable intensity, content title, and playback timecode. An explicitly enabled camera feeds a local disk-based rolling buffer, from which variable-length audience clips are saved with available context and editing handles. A live timeline and Tonight's Highlights support review and deletion. This specification preserves thirteen requirements, defines the development environment and estimated cost, and gives an implementation procedure and verification case for each requirement. It also defines an iterative development process and a Kubernetes-deployable server package. These procedures describe planned behavior, not completed implementation or measured model performance.**

***Index Terms*—audience reactions, acoustic echo cancellation, PC application, server inference, software specification**

I. Introduction

A. Motivation and Scope

Shared laughter or surprise can make a viewing session memorable, but starting a recording after a reaction loses its beginning. Reviewing an entire recording is also inconvenient. Encore Moment identifies candidate audience reactions and retains useful moments for friends, families, and reaction-content creators. Users decide which clips to keep; the system describes observable reactions rather than claiming to infer private emotions.

The first demonstration uses a PC, a microphone, an optional webcam, and a player controlled by the team. The player supplies its decoded playback audio, title, position, and play/pause/seek changes. This makes acoustic echo cancellation (AEC) and scene association implementable without assuming access to third-party streaming applications. Saved video depicts the audience. Native TV integration, mobile clients, and recording the viewed program are outside this implementation baseline.

The PC preprocesses audio, manages sensing controls and a disk-based circular video buffer, and creates saved clips. The AWS server performs audio recognition, facial analysis, event fusion, and event-record management. High-quality audience footage stays on the PC; only analysis audio and reduced-resolution camera frames are transmitted. Local clip storage is the working implementation choice for B, while server inference and the disk buffer follow the recorded project direction. Figure 1 summarizes this processing boundary. Section III identifies development baselines that still require integration validation.

B. Related Software

OBS Studio exposes replay-buffer operations that illustrate retaining recent media and saving it on demand [1]. Encore Moment adds reaction-driven selection, playback context, and sensing controls. YAMNet provides a pretrained sound-event classifier that scores 521 AudioSet classes per audio frame [2], [3]; its labels include the required sound classes. MediaPipe Face Landmarker supplies facial landmarks and blendshape observations [4]. These can support reaction evidence but do not establish an emotional state or speaker identity. GStreamer's WebRTC audio-processing wrapper and segmented recording element provide the initial media building blocks [5], [6]. These sources do not demonstrate the performance of the proposed combined system in our viewing environment.

C. Project Information

This is Submission B, the specification stage of the course project [7]. The repository is https://github.com/Park-chul1/encoremoment. This document's sources are maintained in `submission-b/`. The project topic is PC-based audience reaction capture; the wider viewing-service motivation from Submission A is retained.

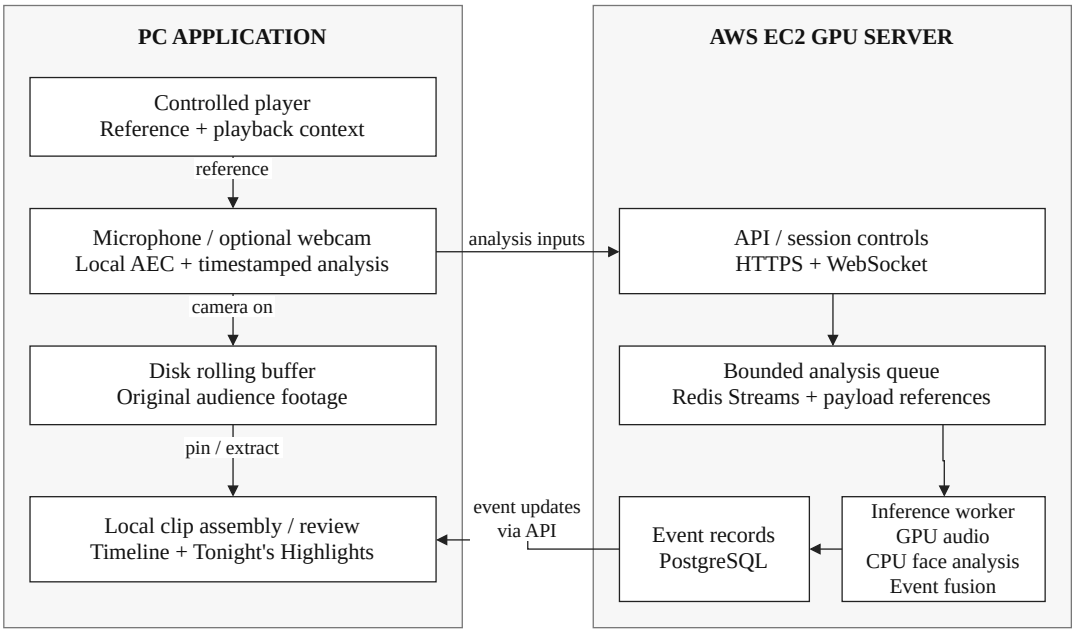


Fig. 1. Planned processing structure. The controlled player supplies playback context as well as the AEC reference. Event updates return through the API; continuous original audience footage remains on the PC. Camera-dependent paths are inactive while the camera is off.

II. Requirements

Saved video means audience footage. The following thirteen requirements define the viewing workflow and server deployment, with corresponding specifications in Section IV.

A. Requirement 1: Echo Cancellation

The system shall use the playback reference to suppress playback sound in microphone audio while retaining audience sound for reaction analysis.

B. Requirement 2: Reaction Recognition

The system shall classify detected audience sounds into at least laughter, scream, and gasp.

C. Requirement 3: Event Records

The system shall record each detected reaction with its type, observed intensity, content title, and timecode. Intensity shall describe the observable reaction rather than an inferred internal emotion.

D. Requirement 4: Highlights

After viewing, the system shall present Tonight's Highlights, prioritizing high-intensity reaction segments and linking any available audience clips for user review.

E. Requirement 5: Camera Activation

The camera shall be off by default and activated only by explicit user action. No video shall be captured, buffered, or uploaded while it is off.

F. Requirement 6: Camera Indication

The interface shall show a persistent indicator while the camera is active and provide a control to stop capture.

G. Requirement 7: Review and Deletion

Users shall be able to review and delete saved reaction clips and their associated media at any time. The interface shall indicate whether a deletion request is pending or completed.

H. Requirement 8: Microphone Control

Users shall be able to disable microphone-based sensing. This shall stop audio capture, upload, and new audio-triggered detections, with the disabled state shown clearly.

I. Requirement 9: Viewing Interface

The interface shall display a real-time reaction timeline, the highlight list, and the current sensing and connection status.

J. Requirement 10: AEC Control

The system shall provide an option to turn echo cancellation on and off and show the selected mode.

K. Requirement 11: Reaction Clips

With the camera enabled, the system shall save an audience clip for each detected reaction, starting 15 s before and ending 10 s after the trigger. Each time the detection threshold is exceeded again before the clip ends, the clip end shall be extended by 5 s, up to a maximum clip length of 60 s. Missing

portions shall be indicated; no footage from before camera activation shall be assumed to exist.

L. Requirement 12: Viewing Sessions

Users shall be able to start and end sensing sessions. The system shall report connection interruptions, avoid duplicate reaction records after reconnection, and clear unsaved temporary media when a session ends.

M. Requirement 13: Server Deployment

The team shall provide server-based AI and computational processing, with the server components deployable on Kubernetes, a platform for managing containerized workloads and services [8].

III. Development Environment

A. Platform, Languages, and Cost

1) PC and server platforms: Linux is the first client target because it offers direct camera/microphone access and a common GStreamer pipeline for controlled playback and capture. Python is selected for application logic and server inference; PySide6 provides the desktop interface, and PyGObject connects the client to GStreamer. Asynchronous acquisition and inference prevent a slow network or model from blocking sensor controls. The reproducible client target is Ubuntu 24.04 LTS, Python 3.12, PySide6 6.8.3, GStreamer 1.24, and FFmpeg 6.1. These are selected integration baselines, not a claim that the application already runs on them.

The inspected workstation runs Ubuntu 26.04.1 LTS on an AMD Ryzen AI 7 350 with 8 cores/16 threads and approximately 14.2 GiB of OS-visible RAM. The client will be tested in its specified environment before distribution; host-installed packages are not the dependency lock. Webcam and microphone models will be recorded during the capture trial. The plan assumes existing equipment, not a new PC or GPU purchase.

The AWS target is one Linux `g4dn.xlarge` in Asia Pacific (Seoul): 4 vCPUs, 16 GiB RAM, and one NVIDIA T4 with 16 GiB GPU memory [9]. An 80 GB encrypted gp3 EBS volume holds the database, model files, and bounded temporary analysis data. Durable records do not depend on instance-store survival. Docker Compose runs the API, queue, database, and inference services on this host. One resident audio model uses the GPU; facial analysis initially uses CPU within the inference service. This preserves server-side audio and video analysis without duplicating GPU model allocations. Load measurements will determine whether concurrency or instance size must change.

2) Software baseline: The server target is Ubuntu 24.04 LTS with separate Python 3.11 environments for the API and inference worker. Proposed pins are FastAPI 0.162.0, TensorFlow 2 with GPU support and the Keras 2 compatibility package required by the YAMNet code [2], MediaPipe 0.10.21, and NumPy 1.26.4 in the worker. Redis 7.2 provides a bounded work stream; PostgreSQL 16 stores sessions, events, result cursors, and deletion state. Docker Engine 28 and Compose 2 are the server packaging baseline. Exact patch versions, image digests, compatible NVIDIA driver/container-toolkit versions, model checksums, and the YAMNet source revision and weight file will be locked after clean installation and an end-to-end inference trial. A dependency lock alone does not prove model or media compatibility.

3) Estimated expenditure: The estimate uses Linux shared-tenancy On-Demand pricing in Seoul, checked November 4, 2026: \$0.647 per GPU-instance hour, \$0.0912 per GB-month of gp3 storage, and \$0.005 per public IPv4 address-hour [10], [11]. For 80 GB-month of EBS and one retained IPv4 address for 720 hours,

$$C(h) = 0.647h + 80(0.0912) + 720(0.005). \tag{1}$$

Thus 40, 80, and 120 active GPU hours cost \$36.78, \$62.66, and \$88.54, respectively, before credits and tax. EBS and an allocated public address can incur charges while the instance is stopped. This excludes paid data transfer beyond applicable allowances, extra storage, paid monitoring, and subsequently purchased devices or licenses.

Existing equipment is budgeted at \$0 incremental hardware cost. No paid development-tool subscription or model-training service is planned. Distribution must retain applicable software and model-license notices; the YAMNet source and pretrained weights keep their upstream license terms [2]. No managed Kubernetes control-plane charge is included. New-account Free Tier credits are intended to offset eligible usage; actual account plan, remaining credit, GPU quota, and expiry are unverified. GPU capability remains in the design even if eligibility requires another funding arrangement [12].

B. Software and Processing Choices

Table I summarizes the selected layers and responsibilities. GStreamer routes decoded player PCM through `webrtcechoprobe` before speaker output, while `webrtcdsp` processes microphone input in the same top-level pipeline. The probe is created before the DSP input becomes active; both branches use the same sample rate [5]. This is local signal preprocessing; reaction inference remains on the server. Camera video is split into independently decodable segments for the disk buffer. A reduced-resolution branch supplies analysis frames. FFmpeg assembles selected local segments into a reviewable clip, without uploading continuous original footage.

The selected audio model is YAMNet, a MobileNet-based classifier with pretrained AudioSet weights and 16 kHz mono input [2], [3]. It scores 0.96 s frames with a 0.48 s hop; these per-frame scores are mapped to capture timestamps. YAMNet is light enough to fall back to CPU inference if GPU capacity is unavailable. “Screaming,” “Laughter,” and “Gasp” map to the required classes. MediaPipe supplies per-face observations [4]; track identifiers are session-local and do not identify people. A single room microphone does not establish speaker-to-face identity. Classification confidence, observable intensity, and facial evidence remain distinct quantities.

TABLE I
SELECTED DEVELOPMENT BASELINE (INTEGRATION VALIDATION PENDING)

Layer	Selected baseline	Responsibility
PC client	Ubuntu 24.04 / Python 3.12 PySide6 6.8.3 / GStreamer 1.24	Player, AEC, sensor controls, disk ring
Clip processing	FFmpeg 6.1 / local SQLite index	Assemble and review audience clips
API / storage	FastAPI 0.162.0 / PostgreSQL 16 Redis 7.2	Sessions, queue, event records
Inference worker	Python 3.11 / TensorFlow 2 YAMNet + MediaPipe 0.10.21	GPU audio, CPU facial analysis, fusion
Deployment	EC2 g4dn.xlarge / Docker Compose Kubernetes-compatible images	Initial host and portable server package

The API accepts timestamped packets and places references to bounded temporary payloads into Redis Streams. Workers acknowledge jobs after event/result transactions commit. Stable keys make retries idempotent; at-least-once delivery is not exactly-once inference [13]. The PC stores its clip index and result cursor in SQLite; PostgreSQL stores server event state. Saved clips remain local in this baseline. Server payloads expire after processing or a bounded retention interval; expiry produces an explicit dropped-input status.

C. Responsibilities and Iterative Process

The existing roles are retained. Soyeon Seol develops the PC interface and represents the User role; Pavel Kim develops backend/session/record services and represents the Customer role; Yujeong Heo develops AEC integration, audio recognition, and intensity evaluation; Chul Park manages development, media buffering/transport, and deployment integration. Facial-analysis integration is planned with AI/media work and needs a named issue owner before implementation. All members review requirement coverage and integration.

GitHub Issues and Projects will manage work beside source and pull requests [14]. The proposed process uses one-week timeboxes, relative estimates, and capacity agreed at planning. This cadence is a project choice, not a lecture-mandated duration. Each issue identifies a Requirement number, acceptance case, owner, reviewer, estimate, and iteration. Board states are Backlog, Ready, In Progress, Review, and Done. Each developer begins with one active implementation task; blocked work is discussed before taking another.

At a brief daily check-in, members report completed work, today's work, blockers, omissions, and useful learning; problem solving follows separately. This follows the lectures' story/task planning, stand-up questions, and continuous integration guidance [15], [16]. Weekly demonstrations with the User and Customer representatives drive reprioritization, followed by a short retrospective. Unfinished work returns to planning. Requirement changes require explicit review rather than silent removal to fit a sprint.

The first increment connects player reference, PC capture, server recognition, and a returned event. Later increments add clips/review, then failure handling and deployment checks. Integration and documentation target the recorded December 3, 2026 development milestone. Weekly scope follows demonstrated capacity. A task is Done when its acceptance case passes, a teammate reviews it, and the merged build works. CI will run unit/contract tests, supported media-fixture tests, container builds, and LaTeX compilation. Hardware/GPU checks run on the corresponding device, with evidence linked to the issue. Expected outcomes for AI-generated code are reviewed independently of that code.

IV. SPECIFICATIONS

All procedures below describe the implementation to be built. The shared packet header contains `session_id`, sensor/control generation, sequence number, capture timestamp, format, and playback epoch. A generation changes when a sensor is disabled/re-enabled; a playback epoch changes on seek or content change. Event IDs remain stable across updates and retries. State-changing requests carry an idempotency key. Invalid or expired inputs return an explicit status rather than silently producing a successful result.

A. Requirement 1: Echo Cancellation

Input/output: microphone PCM and the controlled player's playback reference produce cleaned audience PCM and an AEC status. Both streams use 48 kHz during AEC; output is resampled to the audio model's 16 kHz input. Playback/capture timestamps use the PC's monotonic clock. The playback probe and microphone DSP share one top-level GStreamer pipeline, with the probe initialized first [5].

```
on microphone_block(m, capture_time):
  if microphone_disabled: return
  ref = playback_reference_for(capture_time)
  if AEC_requested and ref.valid:
    y = echo_cancel(m, ref)
    status = ACTIVE
  else:
    y = m
    status = reference_status(AEC_requested)
  enqueue_analysis(resample(y, 16000), status)
```

Playback seek, device change, or timing discontinuity resets alignment and marks AEC as warming up. If the reference is unavailable, the interface reports this and marks uncancelled input; it does not claim echo removal. Automatic gain control and additional noise suppression are disabled in the initial AEC trial to avoid concealing changes in nonverbal reaction strength. Their later use requires preservation checks.

Saved audience footage uses the permitted captured microphone track, not the program's reference audio. *Verification:* Compare AEC-on/off with soundtrack-only, audience-only, and overlapping playback/reaction recordings. Inspect residual playback and whether laughter, scream, and gasp remain recognizable; include seeks and output-device changes.

B. Requirement 2: Reaction Recognition

Input/output: timestamped cleaned audio and, when enabled, camera frames produce labeled reaction intervals. The GPU worker loads YAMNet once and runs overlapping audio windows. Window length, stride, thresholds, and model checksum are versioned configuration. Framewise predictions are approximate temporal evidence, not exact acoustic onset measurements [2].

The initial fusion rule uses high-confidence audio alone, or a lower audio threshold supported by a strong valid facial reaction. For each sound class c, p_c is model confidence, I_a is normalized audio strength, and $V = \max_i v_i$ is the maximum valid per-face reaction score. Multiple weak faces do not accumulate into a strong response. A strong reaction from one person can therefore contribute without requiring a group average. No camera or missing faces means audio-only operation, not a failed session. Facial evidence never relabels a gasp as a sigh or substitutes an inferred emotion for a required sound class.

```
for each aligned analysis_window w:
  if stale_generation(w): discard(w); continue
  p = audio_model(w.audio)
  c = best_of(p, [laughter, scream, gasp])
  V = max_valid_face_score(w.frames, default=0)
  strong = p[c] >= high[c] and Ia(w) >= min_audio
  supported = p[c] >= low[c] and V >= visual_gate
  accepted = strong or supported
  event = update_interval(c, accepted, hysteresis)
  emit_if_changed(event.OPEN, UPDATE, or FINAL)
```

Facial observations use normalized landmark/blendshape changes with pose, visibility, and tracking-validity checks; they do not identify a person's internal state. Open/close hysteresis and a gap tolerance merge adjacent windows of the same class. Different classes remain distinguishable. Ordinary speech, background noise, and uncertainty must pass the same evidence gates rather than automatically becoming an event. Thresholds are calibrated on development recordings and evaluated on separate recordings; no accuracy percentage is asserted here. *Verification:* Use labeled laughter/scream/gasp, speech, soundtrack-only, and mixed recordings. Test camera-off operation, missing/occluded faces, one strong person, and several weak responses. Report per-class errors and measured detection delay.

C. Requirement 3: Event Records

Input/output: an accepted interval and playback anchors produce an upserted event with type, observed intensity, title, and content timecode. Each player anchor records capture time t_0, playback position p_0, playback rate r, play state, and content identity. Within a continuous playing epoch, timecode is $p(t) = p_0 + r(t - t_0)$; pause keeps position fixed. An event crossing a seek/content boundary is split by epoch. Network arrival time never substitutes for capture time.

Audio intensity I_a is the cleaned-audio RMS level relative to the calibrated room baseline, clipped to a configured range. Visual intensity uses the largest valid per-face observation V. The working combined intensity is $I = \max(I_a, V)$ after per-modality normalization. Stored fields retain both values and modality availability; confidence is not used as intensity. This is a ranking signal within a viewing session, not a calibrated physical loudness or emotional score.

```
on event_update(e):
  anchor = get_anchor(e.start, e.epoch)
  if anchor missing: hold_pending(e); return
  record = {e.id, e.type, observed_intensity(e),
            anchor.title, map_time(e, anchor)}
  atomic:
    require current(e) and not deleted(e.id)
    upsert_event(record, key=(session_id, e.id))
    append_result_change(record)
    acknowledge_job()
```

Additional implementation fields include capture start/end, model/configuration version, AEC mode, event revision, and clip availability. Missing playback context is visible as pending and is retried; it is not fabricated. The controlled-player handshake supplies title and anchors before normal sensing begins. *Verification:* Replay the same input/result twice and obtain one event. Check timecodes before/after pause, seek, rate change, and title change, including delayed network delivery.

D. Requirement 4: Highlights

Input/output: a session's committed event records and local clip index produce Tonight's Highlights. Finalized events are sorted by descending observed intensity, with capture time and event ID as deterministic tie-breakers. Events without footage remain reviewable as metadata. Overlapping events may point to one clip without losing their separate event records.

```
on show_highlights(session):
  events = final_undeleted_events(session)
  events.sort_by(-intensity, start, event_id)
  for e in events:
    clip = local_clip_index.lookup(e.id)
    render(e, clip.status, clip.path_if_ready)
  refresh_when_pending_clip_finishes()
```

A clip has pending, ready, partial, unavailable, or deletion-pending status. The UI does not show a playable link until the saved file has been finalized and checked. Highlight generation does not infer that the loudest reaction was the viewer's favorite scene. *Verification:* Use mixed intensities, equal scores, shared clips, camera-off events, failed exports, and pending deletions. Verify ordering and every displayed link.

E. Requirement 5: Camera Activation

Input/output: an explicit enable action in an active session opens the camera and starts a new capture generation. The initial and restored application state is camera-off. Permission denial or device failure keeps it off. The control gate is checked before capture, enqueue, transmission, and final clip commit.

```
on enable_camera_clicked():
  require active_session and device_permission
```

```
  generation = next_camera_generation()
  open_camera(generation)
  if first_frame_received: state = ACTIVE
  else: close_camera(); state = OFF_WITH_ERROR
on disable_camera_clicked():
  close_video_admission_gate()
  invalidate_camera_generation()
  stop_capture_upload(); drop_unsent_video()
  cancel_clip_jobs(); clear_unsaved_video()
  send_control_update(); state = OFF
```

Disabling the camera does not disable audio recognition or delete already saved clips. In-flight server results carry their original generation and cannot authorize new camera capture or resurrect cancelled clips. A later enable starts an empty buffer; it never reuses footage from before activation. *Verification:* Inspect device access, disk writes, and network packets before enable, after deny, and after disable. Race disable against queued upload and clip completion.

F. Requirement 6: Camera Indication

Input/output: actual capture state produces a persistent indicator and an always-available Stop control. The interface distinguishes Starting, Camera on, Stopping, Off, and Error. It reflects device callbacks rather than only the user's requested switch position.

```
on camera_state_changed(actual_state):
  render_persistent_indicator(actual_state)
  show_stop_until_capture_fully_stops()
on stop_clicked():
  invoke_camera_disable_procedure()
on camera_device_lost():
  close_video_admission_gate()
  stop_capture_upload(); cancel_clip_jobs()
  clear_unsaved_video(); render_state(ERROR)
```

The active indicator remains visible while navigating between the timeline and highlights. Camera capture is stopped before the indicator reports Off. Device disconnection cannot leave an apparently active camera or an unreported background recording. *Verification:* Switch views during capture, stop while export is pending, and unplug the webcam. Compare the indicator against actual pipeline state and media writes.

G. Requirement 7: Review and Deletion

Input/output: a selected saved clip is opened locally, or a deletion request removes that clip and its associated media. A persistent deletion operation ID makes retries safe. Local tombstones are written before file deletion and synchronized with the server so delayed work cannot recreate the association.

```
on delete_clip(clip_id):
  op = persist_deletion_intent(clip_id)
  hide_playback_link(); show(PENDING)
  cancel_exports_and_mark_tombstone(clip_id)
  unlink_clip_from_all_associated_events()
  delete_clip_files_thumbnails_and_temp_exports()
  retry_server_media_cleanup(op)
on remote_ack(op) and local_cleanup_done(op):
  mark_completed(op); show(COMPLETED)
```

The baseline does not store completed video in the cloud, but it still cleans associated temporary server payloads and clip references. Reference counts prevent deleting a file still used by a different retained clip; deleting a shared clip removes all links to that same clip. Non-media event metadata may remain with its clip marked deleted. Offline requests stay pending until required remote cleanup is acknowledged. On restart, the client resumes incomplete deletions before permitting export commits. *Verification:* Delete during playback/export, repeat deletion, disconnect during cleanup, and restart the app. Verify that no stale response restores the media or play link.

H. Requirement 8: Microphone Control

Input/output: disabling the microphone closes audio capture, audio upload, and acceptance of new audio-triggered events. Player output can continue for viewing. Camera state is independent; with audio sensing off, this baseline does not create new sound-class events from facial observations alone.

```
on microphone_disable():
  close_audio_admission_gate()
  invalidate_audio_generation()
  stop_microphone(); drop_unsent_audio()
  stop_audio_track_in_future_camera_segments()
  reject_late_audio_results_in_client()
  send_server_control_update()
  render_microphone_disabled()
on worker_before_commit(job):
  require audio_generation_and_session_current()
```

Server cancellation takes effect when the control update is received; during disconnection, expired session leases also reject new commits. The client immediately refuses late results and does not present them as new detections. Re-enabling starts a new generation and resets AEC state. Already saved media remains available for review; recording boundaries preserve whether an audio track was present. *Verification:* Disable during an open event and during a queued inference. Confirm no new accepted audio event, microphone read, or audio upload occurs at the client after the control takes effect.

I. Requirement 9: Viewing Interface

Input/output: ordered result changes and local sensor/connection state produce the timeline, highlight list, and status panel. HTTP endpoints manage sessions, context, controls, events, and deletion acknowledgments. A WSS connection carries bounded analysis messages and event updates; binary payloads include their timestamped headers. Control messages have priority over media. TLS and session-scoped access are implementation mechanisms, not additional numbered requirements.

```
on result(change):
  if wrong_session(change): return
  buffer_by_cursor(change)
  for next in contiguous_after(saved_cursor):
    atomic:
      apply_current_newer_revision(next)
      persist_result_cursor(next.cursor)
  notify_UI_without_blocking_capture()
on connection_lost():
  show(DISCONNECTED); mark_live_results_stale()
```

The screen displays microphone/camera state, requested and effective AEC mode, connection state, and clip/deletion progress. Per-session result cursors are contiguous; a gap triggers replay before the persisted cursor advances. Reconnection resumes after the last committed cursor, with snapshot recovery if the replay window has expired. Sensor controls

remain usable while disconnected. Queue overflow drops stale analysis media with a visible gap status; it never reports stale inference as current live feedback. *Verification:* Deliver out-of-order and duplicated updates, interrupt the socket, and fill the analysis queue. Confirm correct event count, visible connection gaps, and responsive Stop controls.

J. Requirement 10: AEC Control

Input/output: the requested AEC toggle changes the processing mode at an audio-block boundary. Requested mode and effective status are stored separately so "On, reference unavailable" is distinguishable from active cancellation.

```
on set_AEC(requested):
  at_next_audio_boundary:
    mode_revision += 1
    reset_AEC_if_enabling_or_reference_changed()
    apply_echo_cancel_setting(requested)
    stamp_new_blocks(mode_revision,
                     effective_status)
  render_requested_and_effective_modes()
```

A mode change does not disable the microphone or silently stop event recording. A comparison trial processes the same consented test input under both modes and retains the mode revision with evaluation results. Uncancelled playback can produce false reactions; it is shown as an operating condition rather than hidden by the switch. *Verification:* Toggle during playback and overlapping audience sound. Check block labels, the effective-mode display, and behavior when the reference disappears and returns.

K. Requirement 11: Reaction Clips

Input/output: an event's capture interval selects available audience segments from the PC disk ring and produces a local clip plus availability metadata. GStreamer uses closed-GOP encoding and `splitmuxsink`; segment boundaries occur at keyframes and may exceed the requested duration [6]. Only finalized segments enter the index. The application's own eviction policy protects pinned segments; automatic file rotation must not remove them.

Clipping rule. Let t_0 be the capture time at which an event first exceeds its detection threshold. The initial clip interval is $[t_0 - P,\ t_0 + Q]$ with pre-context $P = 15$ s and post-context $Q = 10$ s. An OPEN event immediately pins the available pre-event segments. Each time the threshold is exceeded again (re-fire) at a time not later than the current clip end, the end is extended by $E = 5$ s, capped so that the clip never exceeds the maximum length $L = 60$ s. Re-fires after the clip end start a new clip. FINAL schedules export once the post-context is available. Editing handles H_p, H_q are retained outside this interval and do not count toward L. The rolling buffer must hold at least $P + H_p$ plus the measured trigger delay; a longer delay produces partial coverage that is labeled rather than hidden.

```
on event_OPEN(e):
  start = e.trigger_time - P
  end = e.trigger_time + Q
  pin_available([start-Hp, end+Hq], e.camera_generation)
on threshold_refire(e, t):
  if t <= end:
    end = min(end + E, start + L)
    pin_available([start-Hp, end+Hq], e.camera_generation)
  else: start_new_clip(e, t)
on event_FINAL(e):
  wait_until_post_context_or_capture_boundary(e)
  parts, gaps = finalized_pinned_segments(e.id)
  if no parts: mark_unavailable(e.id); return
  tmp = ffmpeg_assemble_compatible_tracks(parts)
  atomic:
    require job_current(e) and not deleted(e.id)
    verify_decodable(tmp); commit_clip_file(tmp)
    save_index(e.id, coverage, gaps, handles)
  release_pins(); evict_old_unpinned_segments()
```

The editor's default in/out points exclude editing handles while the saved file retains them. Compatible closed-GOP segments can be remuxed; frame-exact cuts or incompatible tracks require transcoding or separate explicitly marked parts. Audio/video synchronization uses capture timestamps. A microphone-state or camera-format change creates a segment boundary. The system neither fabricates missing footage nor silently concatenates incompatible tracks. If reactions continue after the clip reaches L, the next re-fire starts a new linked clip so that the available interval is preserved. At the disk budget limit, unpinned old segments are evicted first; if none can be evicted, capture/export reports storage exhaustion rather than overwriting pinned or saved media. *Verification:* Trigger near camera activation, buffer rollover, and session end; inject network delay, overlapping events, format changes, and disk exhaustion. Check decodability, synchronization, actual coverage, and missing-part labels. Check that a single trigger yields a 25 s clip, that each re-fire extends the end by 5 s, and that no clip exceeds L.

L. Requirement 12: Viewing Sessions

Input/output: Start creates a session and playback handshake; End closes admission and clears unsaved temporary media while retaining previously saved clips and metadata. The camera remains off until explicitly enabled. Start clearly presents microphone sensing and server processing before the user initiates it.

```
on start_clicked():
  session = create_session_with_idempotency_key()
  require player_title_and_anchor_available()
  initialize_generations_and_empty_buffers()
  connect_results(0); enable_requested_audio()
on reconnect():
  sync_controls_then_deletion_intents()
  replay_results_after(last_committed_cursor)
  resend_current_packets_using_original_keys()
on end_clicked():
  close_admission(); invalidate_generations()
  stop_devices_upload(); cancel_unsaved_jobs()
  delete_unsaved_buffers_and_temporary_exports()
  send_end_once(); show_cleanup_status()
```

The server enforces unique packet/job/event keys and commits a result cursor with each record change. Duplicated deliveries update the same event. Late results from a closed session cannot start a new export. On connection loss, the local disk ring remains bounded and the UI shows a gap. If a remote End request cannot arrive, its persisted retry and the server session lease drive cleanup; remote cleanup remains visibly pending until acknowledged. Ending a session cancels unsaved exports rather than promising unavailable post-event footage.

Verification: Repeat Start/End, reconnect after a committed-but-unacknowledged result, end while offline, and restart during cleanup. Check one event per stable ID and absence of unsaved media after cleanup.

M. Requirement 13: Server Deployment

Input/output: versioned API/worker images, configuration, and deployment definitions start server-side processing. Compose is the initial AWS runtime. Kubernetes deployability is retained as a deliverable, not deferred away: the same images will have manifests and a documented smoke test on a GPU-capable Kubernetes environment. A single-node GPU Kubernetes trial may reuse the same AWS host after stopping Compose, avoiding concurrent GPU reservations. A paid EKS control plane is not part of the initial demonstration.

```
build_images_from_locked_dependencies()
validate_config_checksums_and_paths()
start_database_and_queue(); wait_until_healthy()
start_API_and_worker(); load_models_once()
mark_ready_after_dependencies_and_model()
run_fixture_through_complete_server_path()
for target in [Compose, Kubernetes]:
  verify_event_and_idempotent_retry(target)
  restart_worker(); verify_records_survive()
stop_gracefully(); release_leases_and_payloads()
```

The API/worker are separate services; Redis and PostgreSQL are internal services with persistent storage where required. Kubernetes packaging includes Deployments/Services, configuration and secret references, storage claims, readiness checks, and GPU worker resource requests. GPU scheduling requires a compatible host driver, NVIDIA container runtime integration, and device plugin; the worker requests `nvidia.com/gpu: 1` [17], [18]. API availability alone does not mean the inference worker is ready.

One worker initially owns the GPU and bounds in-flight windows. Media payloads use leased temporary storage; a retry cannot reference an already expired payload without reporting the gap. Graceful shutdown stops admission, completes or releases in-flight jobs, and preserves committed database state. Secrets are injected at runtime and are absent from images and the repository. The proposed package includes Compose configuration, Kubernetes manifests, an example configuration, model-download/checksum instructions, and a smoke-test fixture. Actual AWS provisioning and Kubernetes test results are implementation-stage evidence, not outcomes claimed in this specification. *Verification:* Run the fixture on both deployment targets, retry after worker failure, simulate missing GPU/model/storage, and confirm readiness remains false until processing is possible. Record image digests, target versions, outputs, and restart behavior.

REFERENCES

[1] OBS Project, "OBS Studio Frontend API," https://docs.obsproject.com/reference-frontend-api, accessed: Nov. 4, 2026.

[2] TensorFlow Model Garden, "YAMNet," https://github.com/tensorflow/models/tree/master/research/audioset/yamnet, accessed: Nov. 5, 2026.

[3] TensorFlow, "Sound classification with YAMNet," https://www.tensorflow.org/hub/tutorials/yamnet, accessed: Nov. 5, 2026.

[4] Google, "Face Landmarker: Python guide," https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker/python, accessed: Nov. 4, 2026.

[5] GStreamer, "webrtcdsp," https://gstreamer.freedesktop.org/documentation/webrtcdsp/webrtcdsp.html, accessed: Nov. 4, 2026.

[6] ——, "splitmuxsink," https://gstreamer.freedesktop.org/documentation/multifile/splitmuxsink.html, accessed: Nov. 4, 2026.

[7] CSE406 Software Engineering, "Phase 2—Term Project II," Hanyang University, course materials, course assignment handout, Submission B instructions, pp. 2–3. Course copy consulted Nov. 4, 2026.

[8] Kubernetes, "Overview," https://kubernetes.io/docs/concepts/overview/, accessed: Nov. 4, 2026.

[9] Amazon Web Services, "Specifications for accelerated computing instances," https://docs.aws.amazon.com/ec2/latest/instancetypes/ac.html, accessed: Nov. 4, 2026.

[10] ——, "Amazon EC2 regional price list: Asia Pacific (Seoul)," https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/ap-northeast-2/index.json, accessed: Nov. 4, 2026.

[11] ——, "Amazon VPC pricing," https://aws.amazon.com/vpc/pricing/, accessed: Nov. 4, 2026.

[12] ——, "AWS Free Tier plans," https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html, accessed: Nov. 4, 2026.

[13] Redis, "XAUTOCLAIM: stream pending-message recovery," https://redis.io/docs/latest/commands/xautoclaim/, accessed: Nov. 4, 2026.

[14] GitHub, "About Projects," https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects, accessed: Nov. 4, 2026.

[15] CSE406 Software Engineering, "Life-cycle models," Hanyang University, course materials, cse406-lec-03-1-lifecycle.pdf, pp. 45–55, 60. Course copy consulted Nov. 4, 2026.

[16] ——, "Management," Hanyang University, course materials, cse406-lec04-2-management.pdf, pp. 8–12. Course copy consulted Nov. 4, 2026.

[17] Kubernetes, "Schedule GPUs," https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/, accessed: Nov. 4, 2026.

[18] Docker, "Enable GPU support in Docker Compose," https://docs.docker.com/compose/how-tos/gpu-support/, accessed: Nov. 4, 2026.